# REPORT

ON

# Sqoop, Pig, Hive Experiments

(Importing All Tables from MySQL to HDFS using Sqoop)

Submitted in Partial Fulfilment of **MBA**

Program 2025-27

**Course Code: INTM590**

**Mittal School of Business**

**LOVELY PROFESSIONAL UNIVERSITY, PUNJAB**

**APRIL 2026**

**PROJECT BY**


**Name -** Priyanshu Raj

**Reg. No. -** 12522199

**Roll no. –** A23

**Section –** Q5E06

# PART A – Sqoop Experiments

Objective: To understand the concept of storing the data in HDFS using Sqoop.

## Experiment 1: Importing a Single Table from MySQL to HDFS using Sqoop

### 1. Aim

To import a single table (employees) from a MySQL relational database (companydb) into HDFS using the Sqoop import command and verify the data in HDFS.

### 2. Theory

Apache Sqoop (SQL-to-Hadoop) is a command-line tool designed for efficiently transferring bulk data between Apache Hadoop and structured datastores such as relational databases. Sqoop uses MapReduce to import and export data, which provides parallel operation as well as fault tolerance.

The sqoop import command reads rows from a table in an RDBMS and writes them as files in HDFS. It uses JDBC to connect to the database, fetches the schema, generates Java code to parse row data, and then launches a MapReduce job that reads the table in parallel and writes to HDFS as text or Avro/Parquet files.

### 3. Environment / Prerequisites

- Hadoop 2.7.x installed and HDFS services running (NameNode and DataNode).
- Sqoop 1.4.7 installed with MySQL JDBC connector jar in $SQOOP_HOME/lib.
- MySQL running on localhost:3306 with database companydb created.
- Table employees(id, name, designation, salary, dept) created and populated with sample records.
- All commands executed in a Linux terminal as user hduser.

### 4. Step-by-Step Procedure and Commands

### Step 1 – Verify Table Data in MySQL

Open a MySQL shell and check the data in the employees table before import.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 5.7.38 MySQL Community Server (GPL)

mysql> USE companydb;
Database changed
mysql> SELECT * FROM employees LIMIT 5;
+----+--------+-------------+--------+------+
| id | name   | designation | salary | dept |
+----+--------+-------------+--------+------+
|  1 | Rahul  | Engineer    |  55000 | IT   |
|  2 | Anita  | Manager     |  72000 | HR   |
|  3 | Suresh | Analyst     |  48000 | FIN  |
|  4 | Priya  | Developer   |  61000 | IT   |
|  5 | Mohan  | Clerk       |  32000 | ADM  |
+----+--------+-------------+--------+------+
5 rows in set (0.01 sec)

mysql>
```

### Step 2 – Execute Sqoop Import Command

Run the following Sqoop command in the Linux terminal to import the employees table into HDFS.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ sqoop import \
> --connect jdbc:mysql://localhost:3306/companydb \
> --username root -P \
> --table employees \
> --target-dir /user/hduser/employees_sqoop \
> --m 1
Enter password:
26/04/28 10:12:01 INFO sqoop.Sqoop: Running Sqoop version: 1.4.7
26/04/28 10:12:02 INFO tool.CodeGenTool: Beginning code generation
26/04/28 10:12:03 INFO manager.SqlManager: Executing SQL statement:
26/04/28 10:12:03 INFO manager.SqlManager: SELECT t.* FROM `employees` AS t LIMIT 1
26/04/28 10:12:04 INFO orm.CompilationManager: Writing jar file: /tmp/sqoop-hduser/compile/employees.jar
26/04/28 10:12:06 INFO mapreduce.JobSubmitter: number of splits:1
26/04/28 10:12:07 INFO mapreduce.Job: Running job: job_1714297800_0001
26/04/28 10:12:09 INFO mapreduce.Job: Job job_1714297800_0001 running in uber mode: false
26/04/28 10:12:09 INFO mapreduce.Job:  map 0% reduce 0%
26/04/28 10:12:23 INFO mapreduce.Job:  map 100% reduce 0%
26/04/28 10:12:24 INFO mapreduce.Job: Job job_1714297800_0001 completed successfully
26/04/28 10:12:24 INFO mapreduce.Job: Counters: 30
26/04/28 10:12:24 INFO tool.ImportTool: Transferred 0.2215 MB in 22.631 seconds
26/04/28 10:12:24 INFO tool.ImportTool: Retrieved 5 records.
hduser@ubuntu:~$
```

### Step 3 – Verify Imported Data in HDFS

Use HDFS shell commands to list the imported files and preview the content.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ hdfs dfs -ls /user/hduser/employees_sqoop
Found 2 items
-rw-r--r--   1 hduser supergroup          0 2026-04-28 10:12 /user/hduser/employees_sqoop/_SUCCESS
-rw-r--r--   1 hduser supergroup        142 2026-04-28 10:12 /user/hduser/employees_sqoop/part-m-00000

hduser@ubuntu:~$ hdfs dfs -cat /user/hduser/employees_sqoop/part-m-00000
1,Rahul,Engineer,55000,IT
2,Anita,Manager,72000,HR
3,Suresh,Analyst,48000,FIN
4,Priya,Developer,61000,IT
5,Mohan,Clerk,32000,ADM

hduser@ubuntu:~$
```

## 5. Explanation of Commands

--connect: Specifies the JDBC connection URL for the MySQL database. The format is jdbc:mysql://host:port/database_name.

--username root: The MySQL username used for authentication. The -P flag prompts for the password interactively at runtime, which is more secure than passing it in plaintext with --password.

--table employees: Specifies which table to import. Sqoop reads the entire table unless a WHERE clause is specified using --where.

--target-dir /user/hduser/employees_sqoop: The destination HDFS directory where the imported data files will be written. Sqoop creates this directory automatically.

--m 1: Specifies the number of mapper tasks. Using 1 mapper writes a single output file (part-m-00000). More mappers enable parallel reads but require a primary key column for splitting.

### 6. Output / Observation

The HDFS listing shows the employees Sqoop directory containing a _SUCCESS flag file (indicating job completion) and a part-m-00000 file holding the actual CSV data. The cat command shows all five employee records successfully imported with comma-separated fields matching the MySQL table schema.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ hdfs dfs -ls /user/hduser/employees_sqoop
Found 2 items
-rw-r--r--   1 hduser supergroup          0 2026-04-28 10:12 /user/hduser/employees_sqoop/_SUCCESS
-rw-r--r--   1 hduser supergroup        142 2026-04-28 10:12 /user/hduser/employees_sqoop/part-m-00000

hduser@ubuntu:~$ hdfs dfs -cat /user/hduser/employees_sqoop/part-m-00000
1,Rahul,Engineer,55000,IT
2,Anita,Manager,72000,HR
3,Suresh,Analyst,48000,FIN
4,Priya,Developer,61000,IT
5,Mohan,Clerk,32000,ADM

hduser@ubuntu:~$
```

## Experiment 2: Importing All Tables (Except One) from MySQL to HDFS using Sqoop

### 1. Aim

To import all tables from the MySQL database companydb into HDFS using the sqoop import-all-tables command while excluding one specific table (salary) using the --exclude-tables parameter.

### 2. Theory

The sqoop import-all-tables command provides a convenient way to migrate an entire relational database schema to Hadoop in a single operation. Instead of writing a separate import command for each table, Sqoop queries the RDBMS data dictionary to discover all table names and imports each one sequentially.

Each imported table is stored in its own subdirectory under the specified warehouse directory (--warehouse-dir). The --exclude-tables parameter accepts a comma-separated list of table names that should be skipped during the import process, which is useful when certain tables are too large, contain sensitive data, or are not needed in HDFS.

### 3. Environment / Prerequisites

- Hadoop 2.7.x installed and HDFS services running.
- Sqoop 1.4.7 installed with MySQL JDBC connector.
- MySQL database companydb with tables: employees, departments, salary.
- The salary table will be explicitly excluded from the import.

### 4. Step-by-Step Procedure and Commands

#### Step 1 – Execute sqoop import-all-tables Command

Run the following command to import all tables in companydb except the salary table.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ sqoop import-all-tables \
> --connect jdbc:mysql://localhost:3306/companydb \
> --username root -P \
> --warehouse-dir /user/hduser/company_all_tables \
> --m 1 \
> --exclude-tables salary
Enter password:
26/04/28 10:25:01 INFO sqoop.Sqoop: Running Sqoop version: 1.4.7
26/04/28 10:25:03 INFO tool.ImportAllTablesTool: Importing table departments
26/04/28 10:25:04 INFO mapreduce.Job: Running job: job_1714297800_0002
26/04/28 10:25:18 INFO mapreduce.Job:  map 100% reduce 0%
26/04/28 10:25:19 INFO mapreduce.Job: Job job_1714297800_0002 completed successfully
26/04/28 10:25:19 INFO tool.ImportTool: Retrieved 3 records.
26/04/28 10:25:20 INFO tool.ImportAllTablesTool: Importing table employees
26/04/28 10:25:21 INFO mapreduce.Job: Running job: job_1714297800_0003
26/04/28 10:25:35 INFO mapreduce.Job:  map 100% reduce 0%
26/04/28 10:25:36 INFO mapreduce.Job: Job job_1714297800_0003 completed successfully
26/04/28 10:25:36 INFO tool.ImportTool: Retrieved 5 records.
26/04/28 10:25:36 INFO tool.ImportAllTablesTool: [salary] is in the excluded list. Ignoring.
hduser@ubuntu:~$
```

#### Step 2 – Verify Imported Directories in HDFS

List the HDFS warehouse directory to confirm that separate directories were created for each imported table.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ hdfs dfs -ls /user/hduser/company_all_tables
Found 2 items
drwxr-xr-x   - hduser supergroup          0 2026-04-28 10:25 /user/hduser/company_all_tables/departments
drwxr-xr-x   - hduser supergroup          0 2026-04-28 10:25 /user/hduser/company_all_tables/employees

hduser@ubuntu:~$ hdfs dfs -ls /user/hduser/company_all_tables/employees
Found 2 items
-rw-r--r--   1 hduser supergroup          0 2026-04-28 10:25 /user/hduser/company_all_tables/employees/_SUCCE
-rw-r--r--   1 hduser supergroup        142 2026-04-28 10:25 /user/hduser/company_all_tables/employees/part-m

hduser@ubuntu:~$
```

### 5. Explanation of Commands

sqoop import-all-tables: This subcommand automatically discovers all tables in the given database by querying the INFORMATION_SCHEMA and imports each one as a separate MapReduce job.

--warehouse-dir /user/hduser/company_all_tables: Unlike --target-dir which specifies an exact directory for one table, --warehouse-dir specifies a parent directory. Sqoop creates a subdirectory named after each table within this directory.

--exclude-tables salary: Instructs Sqoop to skip the salary table. Multiple tables can be excluded by passing a comma-separated list, for example --exclude-tables salary,logs,audit.

### 6. Output / Observation

The HDFS listing of /user/hduser/company_all_tables shows two subdirectories: departments and employees. The salary table directory is absent, confirming successful exclusion. Each subdirectory contains the _SUCCESS file and part-m-00000 data file for the respective table.


```
hduser@ubuntu: ~
hduser@ubuntu:~$ hdfs dfs -ls /user/hduser/company_all_tables
Found 2 items
drwxr-xr-x   - hduser supergroup          0 2026-04-28 10:25 /user/hduser/company_all_tables/departments
drwxr-xr-x   - hduser supergroup          0 2026-04-28 10:25 /user/hduser/company_all_tables/employees

hduser@ubuntu:~$ hdfs dfs -ls /user/hduser/company_all_tables/employees
Found 2 items
-rw-r--r--   1 hduser supergroup          0 2026-04-28 10:25 /user/hduser/company_all_tables/employees/_SUCCE
-rw-r--r--   1 hduser supergroup        142 2026-04-28 10:25 /user/hduser/company_all_tables/employees/part-m

hduser@ubuntu:~$
```

## Experiment 3: Exporting a Table from HDFS Back to MySQL using Sqoop

### 1. Aim

To export data stored in HDFS back into a MySQL relational database table using the sqoop export command, demonstrating reverse data flow from Hadoop to an RDBMS.

### 2. Theory

The sqoop export command performs the reverse operation of sqoop import. It reads files from HDFS and inserts the rows into a pre-existing table in a relational database. This is commonly used when Hadoop has processed or transformed raw data and the results need to be loaded back into the operational database for application use or reporting.

The target table must already exist in the database with a schema compatible with the data being exported. Sqoop reads the HDFS files in parallel using multiple mappers and performs batch inserts into the RDBMS table. The --export-dir option specifies which HDFS directory contains the source data.

### 3. Step-by-Step Procedure and Commands

#### Step 1 – Create Target Table in MySQL

Before running sqoop export, create the destination table in MySQL with a schema compatible with the HDFS data.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ mysql -u root -p
Enter password:
mysql> USE companydb;
Database changed
mysql> CREATE TABLE employees_export (
    ->   id INT PRIMARY KEY,
    ->   name VARCHAR(50),
    ->   designation VARCHAR(50),
    ->   salary INT,
    ->   dept VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
```

#### Step 2 – Execute Sqoop Export Command

Run the following Sqoop export command to push HDFS data into the MySQL table.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ sqoop export \
> --connect jdbc:mysql://localhost:3306/companydb \
> --username root -P \
> --table employees_export \
> --export-dir /user/hduser/employees_sqoop \
> --m 1
Enter password:
26/04/28 10:38:01 INFO sqoop.Sqoop: Running Sqoop version: 1.4.7
26/04/28 10:38:03 INFO manager.SqlManager: Executing SQL statement:
26/04/28 10:38:03 INFO manager.SqlManager: SELECT t.* FROM `employees_export` AS t LIMIT 1
26/04/28 10:38:05 INFO mapreduce.Job: Running job: job_1714297800_0004
26/04/28 10:38:06 INFO mapreduce.Job: Job job_1714297800_0004 running in uber mode: false
26/04/28 10:38:06 INFO mapreduce.Job:  map 0% reduce 0%
26/04/28 10:38:19 INFO mapreduce.Job:  map 100% reduce 0%
26/04/28 10:38:20 INFO mapreduce.Job: Job job_1714297800_0004 completed successfully
26/04/28 10:38:20 INFO mapreduce.ExportOutputFormat: Transferred 0.2215 MB in 18.341 seconds
26/04/28 10:38:20 INFO tool.ExportTool: Exported 5 records.
hduser@ubuntu:~$
```

### Step 3 – Verify Exported Data in MySQL

Confirm that the records from HDFS have been successfully inserted into the MySQL table.

```
hduser@ubuntu: ~
mysql> SELECT * FROM employees_export;
+----+--------+-------------+--------+------+
| id | name   | designation | salary | dept |
+----+--------+-------------+--------+------+
|  1 | Rahul  | Engineer    |  55000 | IT   |
|  2 | Anita  | Manager     |  72000 | HR   |
|  3 | Suresh | Analyst     |  48000 | FIN  |
|  4 | Priya  | Developer   |  61000 | IT   |
|  5 | Mohan  | Clerk       |  32000 | ADM  |
+----+--------+-------------+--------+------+
5 rows in set (0.00 sec)

mysql>
```

## 5. Output / Observation

The Sqoop export job completed successfully, transferring 5 records from HDFS to MySQL. The MySQL SELECT query confirms all five employee records with their original field values (id, name, designation, salary, dept) are now present in the employees_export table, demonstrating successful reverse data flow from Hadoop to the relational database.

```
hduser@ubuntu: ~
mysql> SELECT * FROM employees_export;
+----+--------+------------+--------+------+
| id | name   | designation | salary | dept |
+----+--------+------------+--------+------+
|  1 | Rahul  | Engineer    |  55000 | IT   |
|  2 | Anita  | Manager     |  72000 | HR   |
|  3 | Suresh | Analyst     |  48000 | FIN  |
|  4 | Priya  | Developer   |  61000 | IT   |
|  5 | Mohan  | Clerk       |  32000 | ADM  |
+----+--------+------------+--------+------+
5 rows in set (0.00 sec)

mysql>
```

LOVELY PROFESSIONAL UNIVERSITY PUNJAB (INDIA)

## PART B – Pig Experiments (10 Marks)

Objective: To implement analytics commands using the tool Apache Pig.

Dataset Used: student_marks.csv — Path: /user/hduser/pigdata/student_marks.csv

Schema: (id:int, name:chararray, subject:chararray, marks:int)

Sample Data:

### Experiment 1: Loading Data into a Bag in Apache Pig

#### 1. Aim

To start the Pig shell and load a CSV dataset into a Pig relation (bag of tuples) using the LOAD command with PigStorage, and verify the schema and contents using DESCRIBE and DUMP.

#### 2. Theory

Apache Pig is a high-level data flow language and execution framework for parallel computation built on top of Hadoop. It abstracts MapReduce programming into a SQL-like scripting language called Pig Latin. Pig processes data as relations, where a relation is a bag (a collection) of tuples, and each tuple is an ordered set of fields.

The LOAD statement reads data from HDFS into a Pig relation. PigStorage is the default storage function that reads delimited text files. The AS clause defines the schema, allowing subsequent operators to reference fields by name and type.

#### 3. Environment / Prerequisites

- Hadoop 2.7.x with HDFS running.
- Apache Pig 0.17.0 installed.
- student_marks.csv uploaded to /user/hduser/pigdata/ in HDFS.
- Commands executed in Pig interactive shell (grunt prompt).

#### 4. Step-by-Step Procedure and Commands

#### Step 1 – Start the Pig Shell

#### Step 2 – Load Data and Verify Schema

```
hduser@ubuntu: ~
hduser@ubuntu:~$ pig
2026-04-28 11:00:14,332 [main] INFO  org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386)
2026-04-28 11:00:14,333 [main] INFO  org.apache.pig.Main - Logging error messages to: /home/hduser/pig_171429
2026-04-28 11:00:15,001 [main] INFO  org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connect
grunt> A = LOAD '/user/hduser/pigdata/student_marks.csv'
>> USING PigStorage(',')
>> AS (id:int, name:chararray, subject:chararray, marks:int);
grunt> DESCRIBE A;
A: {id: int,name: chararray,subject: chararray,marks: int}
grunt> DUMP A;
2026-04-28 11:00:28,441 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(1,Rahul,Maths,85)
(2,Anita,Science,92)
(3,Rahul,Science,75)
(4,Priya,Maths,90)
(5,Suresh,English,68)
(6,Mohan,Science,88)
(7,Deepa,Maths,72)
(8,Kavya,English,95)
grunt>
```

### 5. Explanation of Commands

LOAD '/path/file.csv': Reads the specified file from HDFS into a Pig relation. Pig uses lazy evaluation, so data is not actually read until DUMP or STORE is called.

USING PigStorage(','): Specifies the delimiter. PigStorage is the built-in function for delimited text files. The comma character separates fields.

AS (id:int, name:chararray, ...): Defines the schema for the relation. chararray is Pig's string type. Without a schema, fields can still be accessed by positional index ($0, $1, ...).

DESCRIBE A: Displays the schema of relation A, confirming field names and data types.

DUMP A: Triggers execution and prints all tuples to the console. Each tuple is shown in parentheses with comma-separated field values.

### 6. Output / Observation

DESCRIBE A confirms the schema: A: {id: int, name: chararray, subject: chararray, marks: int}. DUMP A displays all 8 student records as tuples. The data is correctly parsed with integer and string types as defined in the AS clause.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ pig
2026-04-28 11:00:14,332 [main] INFO  org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386)
2026-04-28 11:00:14,333 [main] INFO  org.apache.pig.Main - Logging error messages to: /home/hduser/pig_171429
2026-04-28 11:00:15,001 [main] INFO  org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connect
grunt> A = LOAD '/user/hduser/pigdata/student_marks.csv'
>> USING PigStorage(',')
>> AS (id:int, name:chararray, subject:chararray, marks:int);
grunt> DESCRIBE A;
A: {id: int,name: chararray,subject: chararray,marks: int}
grunt> DUMP A;
2026-04-28 11:00:28,441 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(1,Rahul,Maths,85)
(2,Anita,Science,92)
(3,Rahul,Science,75)
(4,Priya,Maths,90)
(5,Suresh,English,68)
(6,Mohan,Science,88)
(7,Deepa,Maths,72)
(8,Kavya,English,95)
grunt>
```

## Experiment 2: Using the FOREACH Command in Apache Pig

### 1. Aim

To use the FOREACH...GENERATE statement in Pig to apply a transformation on each tuple of a relation, adding a derived column (pass/fail status) based on marks.

### 2. Theory

The FOREACH operator in Pig Latin is used to apply a set of expressions to each tuple in a relation, producing a new relation. It is analogous to the SELECT clause in SQL and the map() operation in functional programming.

The GENERATE keyword specifies the output fields. These can be existing fields from the input relation, computed expressions, or user-defined functions (UDFs). Bincond (the ternary operator ? :) allows conditional logic within GENERATE.

### 4. Step-by-Step Procedure and Commands

### Step 1 – Apply FOREACH to Add a Derived Column

```
hduser@ubuntu: ~
grunt> B = FOREACH A GENERATE name, subject, marks,
>> (marks >= 80 ? 'Pass' : 'Fail') AS status;
grunt> DESCRIBE B;
B: {name: chararray,subject: chararray,marks: int,status: chararray}
grunt> DUMP B;
2026-04-28 11:01:05,112 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(Rahul,Maths,85,Pass)
(Anita,Science,92,Pass)
(Rahul,Science,75,Fail)
(Priya,Maths,90,Pass)
(Suresh,English,68,Fail)
(Mohan,Science,88,Pass)
(Deepa,Maths,72,Fail)
(Kavya,English,95,Pass)
grunt>
```

### 5. Explanation of Commands

FOREACH A GENERATE: Iterates over every tuple in relation A and applies the specified GENERATE expressions to produce relation B.

(marks >= 80 ? 'Pass' : 'Fail') AS status: This is a Pig bincond expression. For each tuple, if marks is 80 or above, the status field is set to 'Pass'; otherwise it is 'Fail'. The result is given the alias status using

The output relation B has 4 fields: name, subject, marks, and the newly derived status. Note that the id field was intentionally dropped to show selective field projection.

### 6. Output / Observation

DUMP B shows 8 tuples, each with name, subject, marks, and status fields. Students with marks >= 80 (Rahul-Maths, Anita, Priya, Mohan, Kavya) show 'Pass' and those below 80 (Rahul-Science, Suresh, Deepa) show 'Fail'. The FOREACH command correctly transformed each tuple by adding the derived column.


```
hduser@ubuntu: ~
grunt> B = FOREACH A GENERATE name, subject, marks,
>> (marks >= 80 ? 'Pass' : 'Fail') AS status;
grunt> DESCRIBE B;
B: {name: chararray,subject: chararray,marks: int,status: chararray}
grunt> DUMP B;
2026-04-28 11:01:05,112 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(Rahul,Maths,85,Pass)
(Anita,Science,92,Pass)
(Rahul,Science,75,Fail)
(Priya,Maths,90,Pass)
(Suresh,English,68,Fail)
(Mohan,Science,88,Pass)
(Deepa,Maths,72,Fail)
(Kavya,English,95,Pass)
grunt>
```

## Experiment 3: Using the GROUP BY Command in Apache Pig

### 1. Aim

To use the GROUP BY operator in Pig to group student records by subject and compute the average marks per subject using FOREACH and AVG function.

### 2. Theory

The GROUP operator in Pig collects tuples that share the same value of a given field into a group. The result is a relation where each tuple contains the group key and a bag of all original tuples that belong to that group.

After grouping, aggregate functions such as COUNT, SUM, AVG, MIN, and MAX can be applied using FOREACH...GENERATE on the grouped relation. This is equivalent to SQL GROUP BY with aggregate functions.

### 4. Step-by-Step Procedure and Commands

#### Step 1 – Group Records by Subject

#### Step 2 – Compute Average Marks Per Subject

```
hduser@ubuntu: ~
grunt> C = GROUP A BY subject;
grunt> DESCRIBE C;
C: {group: chararray,A: {id: int,name: chararray,subject: chararray,marks: int}}
grunt> D = FOREACH C GENERATE
>> group AS subject,
>> AVG(A.marks) AS avg_marks;
grunt> DUMP D;
2026-04-28 11:01:44,771 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(English,81.5)
(Maths,82.33333333333333)
(Science,85.0)
grunt>
```

### 5. Explanation of Commands

GROUP A BY subject: Creates relation C where each tuple has a group key (the subject value) and a nested bag containing all tuples from A with that subject.

DESCRIBE C confirms the schema: C: {group: chararray, A: {id: int, name: chararray, subject: chararray, marks: int}}.

FOREACH C GENERATE group AS subject, AVG(A.marks) AS avg_marks: For each group, projects the subject name and computes the average of all marks values in the nested bag using the built-in AVG function.

### 6. Output / Observation

DUMP D shows three tuples, one per subject: English with average 81.5, Maths with average 82.33, and Science with average 85.0. The GROUP BY command correctly collected all records sharing the same subject and the AVG function computed the mean marks for each group.

```
hduser@ubuntu: ~
grunt> C = GROUP A BY subject;
grunt> DESCRIBE C;
C: {group: chararray,A: {id: int,name: chararray,subject: chararray,marks: int}}
grunt> D = FOREACH C GENERATE
>> group AS subject,
>> AVG(A.marks) AS avg_marks;
grunt> DUMP D;
2026-04-28 11:01:44,771 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(English,81.5)
(Maths,82.33333333333333)
(Science,85.0)
grunt>
```

## Experiment 4: Using the ORDER BY Command in Apache Pig

### 1. Aim

To use the ORDER operator in Pig to sort the aggregated results by average marks in descending order and also sort individual student records by marks in descending order.

### 2. Theory

The ORDER operator in Pig Latin sorts a relation by one or more fields. It is analogous to the SQL ORDER BY clause. The sort direction is specified using ASC (ascending, the default) or DESC (descending).

ORDER performs a total sort of the data across all reducers, ensuring the complete output is in sorted order. This is different from RANK which assigns a rank number to each tuple.

### 4. Step-by-Step Procedure and Commands

### Step 1 – Sort Subjects by Average Marks Descending

### Step 2 – Sort Individual Students by Marks Descending

```
hduser@ubuntu: ~
grunt> E = ORDER D BY avg_marks DESC;
grunt> DUMP E;
2026-04-28 11:02:18,552 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(Science,85.0)
(Maths,82.33333333333333)
(English,81.5)
grunt> E2 = ORDER A BY marks DESC;
grunt> DUMP E2;
2026-04-28 11:02:41,334 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(8,Kavya,English,95)
(2,Anita,Science,92)
(4,Priya,Maths,90)
(6,Mohan,Science,88)
(1,Rahul,Maths,85)
(3,Rahul,Science,75)
(7,Deepa,Maths,72)
(5,Suresh,English,68)
grunt>
```

### 5. Explanation of Commands

ORDER D BY avg_marks DESC: Sorts relation D (which has subject and avg_marks) by the avg_marks field in descending order. This gives Science first (85.0), then Maths (82.33), then English (81.5).

ORDER A BY marks DESC: Sorts the original student relation A by the marks field in descending order, placing the highest scorer (Kavya, 95) first.

ASC is the default sort direction and can be omitted. DESC must be specified explicitly. Multiple sort keys can be used, e.g., ORDER A BY subject ASC, marks DESC.

### 6. Output / Observation

The first DUMP E shows subjects sorted by average marks in descending order: Science (85.0), Maths (82.33), English (81.5). The second DUMP E2 shows all 8 students sorted by individual marks from highest (Kavya, 95) to lowest (Suresh, 68), demonstrating Pig's sorting capability on both aggregated and individual data.

```
hduser@ubuntu: ~
grunt> E = ORDER D BY avg_marks DESC;
grunt> DUMP E;
2026-04-28 11:02:18,552 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(Science,85.0)
(Maths,82.33333333333333)
(English,81.5)
grunt> E2 = ORDER A BY marks DESC;
grunt> DUMP E2;
2026-04-28 11:02:41,334 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(8,Kavya,English,95)
(2,Anita,Science,92)
(4,Priya,Maths,90)
(6,Mohan,Science,88)
(1,Rahul,Maths,85)
(3,Rahul,Science,75)
(7,Deepa,Maths,72)
(5,Suresh,English,68)
grunt>
```

## Experiment 5: Using the FILTER Command in Apache Pig

### 1. Aim

To use the FILTER operator in Pig to select only those student records where marks are greater than or equal to 80, demonstrating conditional row filtering.

### 2. Theory

The FILTER operator in Pig Latin selects tuples from a relation based on a boolean condition. It is analogous to the SQL WHERE clause and the filter() operation in functional programming.

FILTER is applied to an entire relation and produces a new relation containing only those tuples for which the specified condition evaluates to true. The condition can use comparison operators (==, !=, >, <, >=, <=), logical operators (AND, OR, NOT), and pattern matching (MATCHES).

### 4. Step-by-Step Procedure and Commands

#### Step 1 – Apply FILTER to Select High Scorers

```
hduser@ubuntu: ~
grunt> F = FILTER A BY marks >= 80;
grunt> DUMP F;
2026-04-28 11:03:10,228 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(1,Rahul,Maths,85)
(2,Anita,Science,92)
(4,Priya,Maths,90)
(6,Mohan,Science,88)
(8,Kavya,English,95)
grunt>
```

### 5. Explanation of Commands

FILTER A BY marks >= 80: Evaluates the condition marks >= 80 for every tuple in relation A. Only tuples where this condition is true are included in the output relation F.

The condition marks >= 80 uses the >= (greater than or equal to) comparison operator. The result F is a new relation with the same schema as A but containing only the matching tuples.

For more complex filtering, multiple conditions can be combined, for example: FILTER A BY marks >= 80 AND subject == 'Maths'. FILTER does not change the schema; it only reduces the number of tuples.

### 6. Output / Observation

DUMP F shows 5 tuples: Rahul (Maths, 85), Anita (Science, 92), Priya (Maths, 90), Mohan (Science, 88), and Kavya (English, 95). The 3 students with marks below 80 (Rahul-Science 75, Suresh 68, Deepa 72) are correctly excluded from the output.


```
hduser@ubuntu: ~
grunt> F = FILTER A BY marks >= 80;
grunt> DUMP F;
2026-04-28 11:03:10,228 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLa
(1,Rahul,Maths,85)
(2,Anita,Science,92)
(4,Priya,Maths,90)
(6,Mohan,Science,88)
(8,Kavya,English,95)
grunt>
```

## PART C – Hive Experiments (20 Marks)

Objective: To implement analytics commands using the tool Apache Hive.

Dataset Used: students.csv — Local path: /home/hduser/data/students.csv

Schema: (id INT, name STRING, dept STRING, marks INT)

Sample Data:

### Experiment 1: Creating an Internal Table in Hive and Loading Data from Local System

#### 1. Aim

To create an internal (managed) table in Apache Hive within a new database, load data from the local filesystem into the table, and query the data using SELECT.

#### 2. Theory

Apache Hive is a data warehouse infrastructure built on top of Hadoop for providing data summarization, query, and analysis using a SQL-like language called HiveQL. Hive translates SQL queries into MapReduce or Tez jobs that execute on the Hadoop cluster.

An internal table (also called a managed table) is a table where Hive fully manages both the metadata (stored in the Hive Metastore) and the data (stored in HDFS under the Hive warehouse directory). When an internal table is dropped, both the metadata and the underlying HDFS data are permanently deleted. This is in contrast to external tables where only the metadata is removed on DROP.

#### 3. Environment / Prerequisites

- Hadoop 2.7.x installed and HDFS services running.
- Apache Hive 2.3.9 installed with Hive Metastore configured.
- Local file /home/hduser/data/students.csv created with sample student data.
- Hive shell started using the hive command in the Linux terminal.

#### 4. Step-by-Step Procedure and Commands

#### Step 1 – Create Hive Database

```
hduser@ubuntu: ~
hduser@ubuntu:~$ hive

Logging initialized using configuration in jar:file:/opt/hive/lib/hive-common-2.3.9.jar!/hive-log4j2.properti
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions.
Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.

hive> CREATE DATABASE IF NOT EXISTS ca1db;
OK
Time taken: 0.812 seconds
hive> USE ca1db;
OK
Time taken: 0.014 seconds
hive>
```

### Step 2 – Create Internal Table and Load Data

```
hduser@ubuntu: ~
hive> CREATE TABLE IF NOT EXISTS students_internal (
    >   id INT,
    >   name STRING,
    >   dept STRING,
    >   marks INT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE;
OK
Time taken: 0.523 seconds
hive> LOAD DATA LOCAL INPATH '/home/hduser/data/students.csv'
    > INTO TABLE students_internal;
Loading data to table ca1db.students_internal
OK
Time taken: 0.391 seconds
hive>
```

### Step 3 – Query and Verify Data

```
hduser@ubuntu: ~
hive> SELECT * FROM students_internal;
OK
1	Rahul	CSE	85
2	Anita	ECE	90
3	Suresh	ME	78
4	Priya	CSE	92
5	Mohan	EEE	65
Time taken: 0.148 seconds, Fetched: 5 row(s)
hive>
```

### 5. Explanation of Commands

CREATE DATABASE IF NOT EXISTS ca1db: Creates a new Hive database named ca1db. The IF NOT EXISTS clause prevents an error if the database already exists.

CREATE TABLE ... ROW FORMAT DELIMITED FIELDS TERMINATED BY ',': Creates a new table definition in the Hive Metastore. ROW FORMAT DELIMITED specifies that each row in the file represents one record, and FIELDS TERMINATED BY ',' indicates comma-separated values.

STORED AS TEXTFILE: Specifies the underlying file format in HDFS. TEXTFILE is the simplest and most human-readable format. Other options include ORC, Parquet, and Avro for better compression and performance.

LOAD DATA LOCAL INPATH '/home/hduser/data/students.csv' INTO TABLE students_internal: The LOCAL keyword tells Hive the file is on the local filesystem (not HDFS). Hive copies the file into the HDFS warehouse directory for students_internal. Without LOCAL, Hive would move the file from an HDFS path.

### 6. Output / Observation

The CREATE TABLE command completes with OK status. The LOAD DATA command copies students.csv from the local filesystem into the Hive warehouse directory. The SELECT * query returns all 5 student records with tab-separated output, confirming successful data load.

```
hduser@ubuntu: ~
hive> SELECT * FROM students_internal;
OK
1	Rahul	CSE	85
2	Anita	ECE	90
3	Suresh	ME	78
4	Priya	CSE	92
5	Mohan	EEE	65
Time taken: 0.148 seconds, Fetched: 5 row(s)
hive>
```

## Experiment 2: Creating an External Table in Hive and Loading Data from HDFS

### 1. Aim

To create an external Hive table that points to an existing HDFS location, demonstrating that Hive only manages the metadata while the data remains at the specified HDFS path.

### 2. Theory

An external table in Hive is created with the EXTERNAL keyword. Unlike an internal table, Hive does not manage the underlying data for an external table. The data remains at the HDFS LOCATION specified during table creation. When an external table is dropped, only the metadata in the Hive Metastore is removed; the actual data files in HDFS remain intact.

External tables are preferred in production environments where the data is shared between multiple systems or tools, where data persistence beyond the table lifecycle is required, and where data is owned by a team different from the one managing the Hive schema.

### 3. Environment / Prerequisites

- Hadoop HDFS running with hduser account.
- Apache Hive 2.3.9 with ca1db database created from Experiment C.1.
- students.csv available on local filesystem at /home/hduser/data/students.csv.

### 4. Step-by-Step Procedure and Commands

#### Step 1 – Upload Data File to HDFS External Location

```
hduser@ubuntu: ~
hive> exit;
hduser@ubuntu:~$ hdfs dfs -mkdir -p /user/hduser/hive_external/students_ext
hduser@ubuntu:~$ hdfs dfs -put /home/hduser/data/students.csv /user/hduser/hive_external/students_ext/
hduser@ubuntu:~$ hdfs dfs -ls /user/hduser/hive_external/students_ext
Found 1 items
-rw-r--r--   1 hduser supergroup         95 2026-04-28 11:30 /user/hduser/hive_external/students_ext/students
hduser@ubuntu:~$
```

### Step 2 – Create External Table Pointing to HDFS Location

```
hduser@ubuntu: ~
hduser@ubuntu:~$ hive
hive> USE ca1db;
OK
Time taken: 0.011 seconds
hive> CREATE EXTERNAL TABLE IF NOT EXISTS students_external (
    >   id INT,
    >   name STRING,
    >   dept STRING,
    >   marks INT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE
    > LOCATION '/user/hduser/hive_external/students_ext';
OK
Time taken: 0.347 seconds
hive> SELECT * FROM students_external;
OK
1□Rahul□CSE□85
2□Anita□ECE□90
3□Suresh□ME□78
4□Priya□CSE□92
5□Mohan□EEE□65
Time taken: 0.127 seconds, Fetched: 5 row(s)
hive>
```

### 5. Explanation of Commands

EXTERNAL keyword: Tells Hive that this is an external table. Hive will only store the schema definition in the Metastore; it will not move, copy, or manage the data files.

LOCATION '/user/hduser/hive_external/students_ext': Specifies the HDFS directory where the data files are stored. Hive reads all files in this directory when the table is queried. This directory must already contain the data files before querying.

Key difference from internal tables: Running DROP TABLE students_external removes only the Hive table definition. The HDFS files at the LOCATION path remain untouched and can still be accessed directly via HDFS or used by another Hive table definition.

No LOAD DATA is needed: Since the data is already in HDFS, Hive simply creates a logical mapping. Any new files added to the LOCATION directory are automatically visible through the table.

### 6. Output / Observation

The external table is created successfully. The SELECT * query returns all 5 student records by reading the students.csv file from the HDFS LOCATION path. Unlike the internal table, if DROP TABLE students_external were executed, the HDFS data at /user/hduser/hive_external/students_ext would remain intact.

```
hduser@ubuntu: ~
hduser@ubuntu:~$ hive
hive> USE ca1db;
OK
Time taken: 0.011 seconds
hive> CREATE EXTERNAL TABLE IF NOT EXISTS students_external (
    >   id INT,
    >   name STRING,
    >   dept STRING,
    >   marks INT
    > )
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE
    > LOCATION '/user/hduser/hive_external/students_ext';
OK
Time taken: 0.347 seconds
hive> SELECT * FROM students_external;
OK
1□Rahul□CSE□85
2□Anita□ECE□90
3□Suresh□ME□78
4□Priya□CSE□92
5□Mohan□EEE□65
Time taken: 0.127 seconds, Fetched: 5 row(s)
hive>
```

## Experiment 3: Static and Dynamic Partitioning in Hive

### 1. Aim

To implement both static and dynamic partitioning in Hive, demonstrating how partitioning organizes data into subdirectories for efficient query performance.

### 2. Theory

Partitioning in Hive is a technique to divide a table into related parts based on the values of a particular column. Each partition is stored as a separate subdirectory in HDFS. When a query filters on the partition column, Hive only reads the relevant partition directories (partition pruning), dramatically reducing I/O.

Static partitioning requires the user to explicitly specify the partition value in the LOAD DATA or INSERT statement. This is efficient when the partition value is known at data load time. Dynamic partitioning, on the other hand, automatically creates partition directories based on the data values during an INSERT...SELECT operation. It is useful when migrating data from a staging table where partition values vary across records.

### 3. Environment / Prerequisites

- Hadoop HDFS running.
- Hive 2.3.9 with ca1db database.
- Local file /home/hduser/data/sales_2024.csv (format: order_id,amount) for static partition.
- Local file /home/hduser/data/sales_all.csv (format: order_id,amount,year) for dynamic partition.

### C.3.1 – Static Partitioning

### 4. Step-by-Step Procedure and Commands

### Step 1 – Create Partitioned Table and Load with Static Partition

```
hduser@ubuntu: ~
hive> CREATE TABLE sales_static (
    >   order_id INT,
    >   amount DOUBLE
    > )
    > PARTITIONED BY (year INT)
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE;
OK
Time taken: 0.412 seconds
hive> LOAD DATA LOCAL INPATH '/home/hduser/data/sales_2024.csv'
    > INTO TABLE sales_static
    > PARTITION (year=2024);
Loading data to table ca1db.sales_static partition (year=2024)
OK
Time taken: 0.318 seconds
hive> SELECT * FROM sales_static WHERE year = 2024;
OK
101□5000.0□2024
102□7000.0□2024
Time taken: 0.201 seconds, Fetched: 2 row(s)
hive>
```

### C.3.2 – Dynamic Partitioning

### 4. Step-by-Step Procedure and Commands

### Step 1 – Create Staging Table

### Step 2 – Enable Dynamic Partitioning and Insert

```
hduser@ubuntu: ~
hive> SET hive.exec.dynamic.partition = true;
hive> SET hive.exec.dynamic.partition.mode = nonstrict;
hive> CREATE TABLE sales_dynamic (
    >   order_id INT,
    >   amount DOUBLE
    > )
    > PARTITIONED BY (year INT)
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE;
OK
Time taken: 0.392 seconds
hive> INSERT OVERWRITE TABLE sales_dynamic
    > PARTITION (year)
    > SELECT order_id, amount, year FROM sales_all;
OK
Time taken: 25.831 seconds
hive> SHOW PARTITIONS sales_dynamic;
OK
year=2023
year=2024
Time taken: 0.113 seconds, Fetched: 2 row(s)
hive> SELECT * FROM sales_dynamic WHERE year = 2023;
OK
201□8000.0□2023
Time taken: 0.189 seconds, Fetched: 1 row(s)
hive>
```

### 5. Explanation of Commands

PARTITIONED BY (year INT): Defines year as the partition column. Hive creates a separate HDFS subdirectory for each distinct year value (e.g., .../sales_static/year=2024/).

Static partition – PARTITION (year=2024): The user explicitly specifies the partition value in the LOAD DATA statement. This is static because the value is hardcoded. Hive moves the file into the year=2024 subdirectory.

SET hive.exec.dynamic.partition = true: Enables dynamic partitioning mode. By default it is disabled.

SET hive.exec.dynamic.partition.mode = nonstrict: In nonstrict mode, Hive allows all partition columns to be dynamic. In strict mode (default), at least one partition column must be static.

INSERT OVERWRITE TABLE sales_dynamic PARTITION (year) SELECT order_id, amount, year FROM sales_all: Hive automatically reads the year column value from each row of sales_all and creates or overwrites the corresponding year=XXXX partition directory in HDFS. This is dynamic because Hive determines partition values at runtime from the data.

SHOW PARTITIONS sales_dynamic: Lists all partition directories that have been created for the table, confirming year=2023 and year=2024 both exist.

### 6. Output / Observation

Static partitioning: The SELECT * FROM sales_static WHERE year = 2024 query returns 2 records (order_id 101 and 102). Hive reads only the year=2024 partition directory, skipping other partitions. Dynamic partitioning: SHOW PARTITIONS sales_dynamic confirms two partitions were created automatically: year=2023 and year=2024. The SELECT with WHERE year = 2023 returns order 201 (amount 8000), demonstrating correct automatic partition routing.

```
hduser@ubuntu: ~
hive> SET hive.exec.dynamic.partition = true;
hive> SET hive.exec.dynamic.partition.mode = nonstrict;
hive> CREATE TABLE sales_dynamic (
    >   order_id INT,
    >   amount DOUBLE
    > )
    > PARTITIONED BY (year INT)
    > ROW FORMAT DELIMITED
    > FIELDS TERMINATED BY ','
    > STORED AS TEXTFILE;
OK
Time taken: 0.392 seconds
hive> INSERT OVERWRITE TABLE sales_dynamic
    > PARTITION (year)
    > SELECT order_id, amount, year FROM sales_all;
OK
Time taken: 25.831 seconds
hive> SHOW PARTITIONS sales_dynamic;
OK
year=2023
year=2024
Time taken: 0.113 seconds, Fetched: 2 row(s)
hive> SELECT * FROM sales_dynamic WHERE year = 2023;
OK
201□8000.0□2023
Time taken: 0.189 seconds, Fetched: 1 row(s)
hive>
```

## Overall Conclusion / Learning Outcomes

Practical report demonstrates the implementation of three major Big Data tools in the Hadoop ecosystem: Apache Sqoop, Apache Pig, and Apache Hive.

### Part A – Apache Sqoop

Sqoop was successfully used to perform bidirectional data transfer between a MySQL relational database and HDFS. The import command transferred individual and all-tables data from MySQL into HDFS using MapReduce, while the export command demonstrated reverse data flow from HDFS back to MySQL. These experiments validated Sqoop as an essential ETL tool in Big Data pipelines.

### Part B – Apache Pig

Apache Pig was used to perform data analytics operations on a student marks dataset using Pig Latin. The LOAD, FOREACH, GROUP BY, ORDER BY, and FILTER operations demonstrated Pig's ability to process large datasets with simple, readable scripts that are automatically compiled into MapReduce jobs by the Pig engine.

### Part C – Apache Hive

Hive was used to implement structured data warehousing on HDFS. Internal tables were shown to manage full data lifecycle, while external tables demonstrated schema-on-read with data persistence beyond table lifetime. Static and dynamic partitioning demonstrated how Hive organizes data to enable efficient partition-pruned queries on large datasets.

Through this CA, hands-on experience was gained in setting up Hadoop-based data pipelines, executing SQL-like analytics without writing Java MapReduce code, and understanding the trade-offs between different data management strategies in a distributed computing environment.